REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Isidora Sofía Villegas Hernández

9 de abril de 2026

01:30

Resumen

En el presente informe se realiza un análisis experimental de algoritmos de ordenamiento (`MergeSort`[2], `QuickSort`[3], `StdSort`) y multiplicación de matrices (`Naive`, `Strassen`[4]), con el objetivo de evaluar su rendimiento en la práctica y comparar los resultados obtenidos con las predicciones teóricas basadas en la notación asintótica. Se implementaron diversos algoritmos tanto de ordenamiento como para multiplicación de matrices, se ejecutaron y se recopilaron datos sobre su tiempo de ejecución y uso de recursos, los cuales fueron graficados principalmente en escala logarítmica para una mejor visualización. Este análisis permite identificar las diferencias entre el comportamiento teórico y real de los algoritmos, así como comprender mejor las implicaciones prácticas de sus complejidades computacionales.

1. Entorno experimental

Los experimentos fueron realizados en una máquina virtual con las siguientes características:

- **Hardware:** (máquina física)
  - Procesador: Intel Core i5-10600K @ 4.10GHz
  - Memoria Ram: 16GB DDR4 2666MHz
  - GPU: NVIDIA GeForce RTX 3060ti
  - Almacenamiento: 1TB SSD M.2 NVMe PCIe 3.0
- **Detalles Máquina Virtual:**
  - Sistema Operativo: Ubuntu 24.04.4 LTS

- Compilador: g++ 13.3.0
- Recursos Asignados a Máquina Virtual: 12 procesadores lógicos (hilos) y 7.7 GiB de memoria RAM.

Mediciones

1. **Tiempo de ejecución:** Se utilizó la biblioteca <chrono>de C++ para medir el tiempo transcurrido en ms durante la ejecución de cada algoritmo individualmente. Esto excluyendo operaciones de entrada/salida y compilación. Para la ejecución de los algoritmos de ordenamiento se realizaron 2 ejecuciones completas correspondientes al dataset generado. Los resultados de estas ejecuciones mostraron conscistencia entre sí (fig. 5), lo que indica que el rendimiento de los algoritmos es estable y predecible bajo las condiciones de prueba establecidas.

2. **Consumo de memoria:** Se empleó getrusage() para medir el consumo máximo residente (RSS) durante la ejecución de cada algoritmo. En el caso particular de los algoritmos de ordenamiento, se midió en dos ejecuciones, inicialmente solo medía el valor máximo RSS alcanzado por el proceso completo, no necesariamente la memoria extra usada por el algoritmo, por lo que se decidió medir el consumo de memoria antes y después de la ejecución del algoritmo, y restar ambos valores para obtener una estimación más precisa del consumo de memoria adicional.

3. **Compilación:** Cada medición se estandarizó con la utilización de makefiles para compilar los algoritmos con las mismas opciones de optimización (-O3) y sin incluir información de depuración (-g). Esto garantiza que las comparaciones entre algoritmos sean justas y consistentes.

4. **Casos de prueba:** Se utilizaron sólo los datasets generados por los scripts correspondientes a cada sección.

Datasets

Los datasets utilizados para los experimentos fueron generados utilizando scripts específicos para cada sección. Estos scripts se encargaron de crear conjuntos de datos con dos tipos de dominios: `D1`: {0,1}, `D7`: {0,1,2,3,4,5,6,7,8,9} en donde para algoritmos de ordenamiento la estructura de pruebas son arreglos de tipo `ascendente`, `descendente` y `aleatorio` dentro de los rangos de valores de 10^3 hasta 10^7 y para multiplicación de matrices de los tipos `dispersa`, `diagonal` y `densa` dentro de los rangos de valores de 2^2 hasta 2^{10}. Estos datasets proporcionaron una base sólida para evaluar el rendimiento de los algoritmos en diferentes escenarios y condiciones, permitiendo así obtener conclusiones más robustas sobre su comportamiento.

2. Resultados y Análisis

Algoritmos de ordenamiento - Tiempo de ejecución

Los algoritmos de ordenamiento muestran patrones de rendimiento diferentes a las expectativas teóricas, revelando la complejidad de implementar algoritmos eficientes en entornos reales.

- **Quicksort:** Aunque teóricamente tiene un tiempo de ejecución de $O(n \log n)$ en el mejor y promedio caso, en la práctica, su rendimiento puede verse afectado por la elección del pivote. En casos donde el pivote no se selecciona adecuadamente (por ejemplo, si el arreglo ya está ordenado), el tiempo de ejecución puede degradarse a $O(n^2)$. Dado que la implementación utilizada selecciona un pivote aleatorio pero ciertos casos pueden no ser tan aleatorios, se observa que el rendimiento de Quicksort puede variar significativamente, especialmente con entradas grandes o patrones reiterativos de datos. (fig. 6)

- **Mergesort:** En la ejecución de este algoritmo se dieron resultados consistentes con el tiempo de ejecución teórico de $O(n \log n)$, A diferencia de algoritmos sensibles a la estructura de la entrada, Mergesort garantiza un rendimiento invariable gracias a su estricto enfoque de dividir y conquistar; al particionar siempre el arreglo en mitades exactas y ejecutar una fase de combinación eficiente, evita por completo la degradación a casos más complejos. (fig. 7)

- **Sort (std::sort en C++):** La función `std::sort` de C++ se basa en un algoritmo híbrido llamado Introsort, que combina `Quicksort`, `Heapsort` y `Mergesort`. En la práctica, std::sort mostró un rendimiento consistente con su tiempo de ejecución teórico de $O(n \log n)$ en el mejor, promedio y peor caso. Esto se debe a que Introsort adapta su estrategia de ordenamiento según las características de los datos, lo que le permite mantener un rendimiento eficiente independiente de la naturaleza de los datos de entrada. (fig. 8)

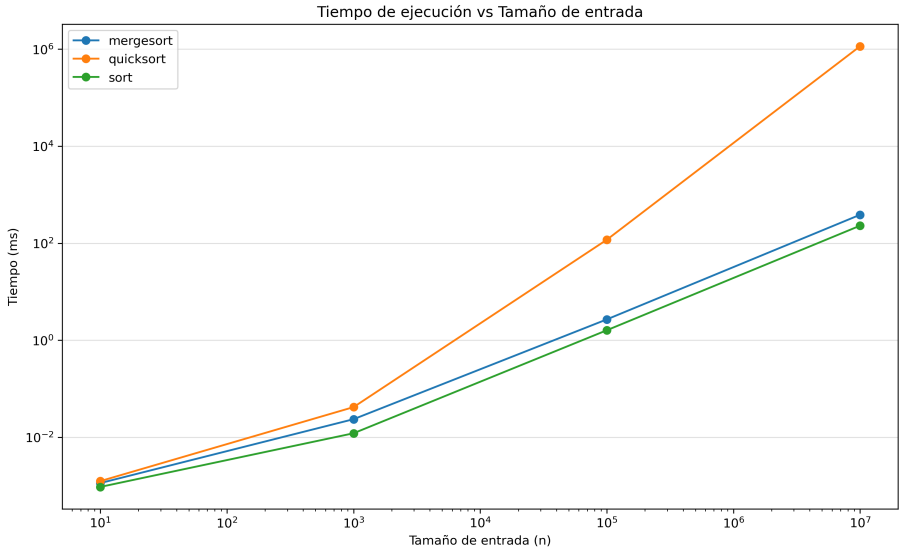

Figura 1: Comparación de tiempos de ejecución entre algoritmos de ordenamiento.

Algoritmos de ordenamiento - Análisis de memoria

En cuanto al consumo de memoria, los algoritmos de ordenamiento presentan diferencias significativas en su uso de recursos, lo que se refleja en las mediciones realizadas. Como ya se mencionó, para obtener una estimación más precisa del consumo de memoria adicional, se midió el consumo de memo-

ria antes y después de la ejecución del algoritmo, y se restaron ambos valores. Particularmente para cada algoritmo se observaron los siguientes patrones:

- **Quicksort:** El bajo y estable consumo de memoria registrado para Quicksort confirma su eficiencia espacial teórica de $O(\log n)$. Este comportamiento fundamentalmente por su diseño de ordenamiento in-place, el cual realiza los intercambios directamente sobre el vector original y no requiere de la asignación dinámica de arreglos auxiliares. A su vez, se realizó una modificación a la implementación de Quicksort para acotar la profundidad de recursión, lo que contribuyó a mantener un consumo de memoria constante incluso en casos de entradas grandes.

- **Mergesort:** Durante la evaluación empírica del algoritmo Mergesort, se registró un aumento significativo en el peak de memoria residente (RSS). Este comportamiento experimental es consistente con la base teórica. El procedimiento estándar no opera de manera in-place, ya que requiere la asignación explícita de arreglos auxiliares para almacenar temporalmente los elementos durante la etapa de fusión. Esto genera un requerimiento de memoria adicional estrictamente proporcional a la cantidad de datos, resultando en una complejidad espacial de $O(n)$. [1]

- **Sort (std::sort en C++):** Utilizando el algoritmo ya implementado, las mediciones iniciales de std::sort arrojaron un consumo de memoria atípicamente alto. El análisis detallado del entorno de pruebas muestra que el incremento no es a raíz del algoritmo, sino un artefacto de la implementación del wrapper utilizado en C++. Al definir la función invocadora con un retorno por valor sobre un contenedor std::vector pasado por referencia, el estándar del lenguaje exige la ejecución del constructor de copia (Copy Constructor) para retornar el resultado. Este proceso fuerza la creación temporal de un bloque de memoria para duplicar los n elementos, elevando el peak de memoria a $O(n)$ en la reasignación, lectura que es capturada por la función del sistema operativo (getrusage()). [5]

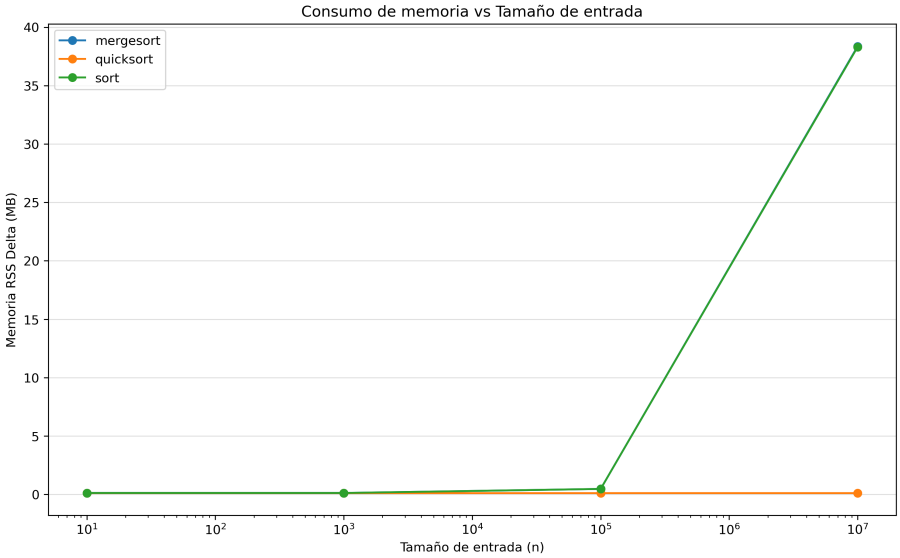

Figura 2: Comparación de uso de memoria entre algoritmos de ordenamiento.

Esta gran diferencia en el consumo de memoria entre los algoritmos se evidencia claramente, la diferencia significativa entre `Quicksort` y los otros dos algoritmos son coherentes con las expectativas en base a las implementaciones utilizadas, siendo `Mergesort` y `Sort` los que presentan un consumo de memoria adicional considerablemente mayor, los que a su vez tienen una diferencia de uso de memoria despreciable, lo que se puede atribuir a la naturaleza de las implementaciones y a la forma en que gestionan la memoria, como se explicó anteriormente. (fig. 9)

Multiplicación de Matrices - Tiempo de ejecución

Las ejecuciones de las multiplicaciones de matrices presentaron un comportamiento que se alinea con las expectativas teóricas, las cuales son:

- **Algoritmo Naive:** Este es el método clásico utilizando tres bucles anidados, el cual tiene un tiempo de ejecución teórico de $O(n^3)$. Durante los experimentos este algoritmo muestra un crecimiento esperado evidente; para matrices de tamaño moderado a pequeño, su simplicidad y las nulas llamadas recursivas permiten que se ejecute rápidamente, pero su rendimiento decae drásticamente frente a entradas de gran magnitud. (fig. 12)

- **Algoritmo de Strassen:** Este algoritmo utiliza el paradigma de dividir y conquistar para reducir la cantidad de multiplicaciones necesarias en cada paso, logrando un tiempo de ejecución teórico optimizado de $O(n^{\log_2 7})$, lo que equivale aproximadamente a $O(n^{2,81})$ [6]. Sin embargo, en la práctica solo es más rápido que el algoritmo cuando las matrices son muy grandes. Esto se debe a que la reducción en multiplicaciones exige un aumento considerable en las sumas y restas de submatrices temporales, generando una sobrecarga constante (overhead) que consume el tiempo de ejecución en entradas pequeñas. Dentro de la implementación se pudo modificar el rango de tamaño de matriz donde se aplicaba Strassen o Naive como auxiliar, pero dado el fin de este análisis se utilizó sólo Strassen. (fig. 13)

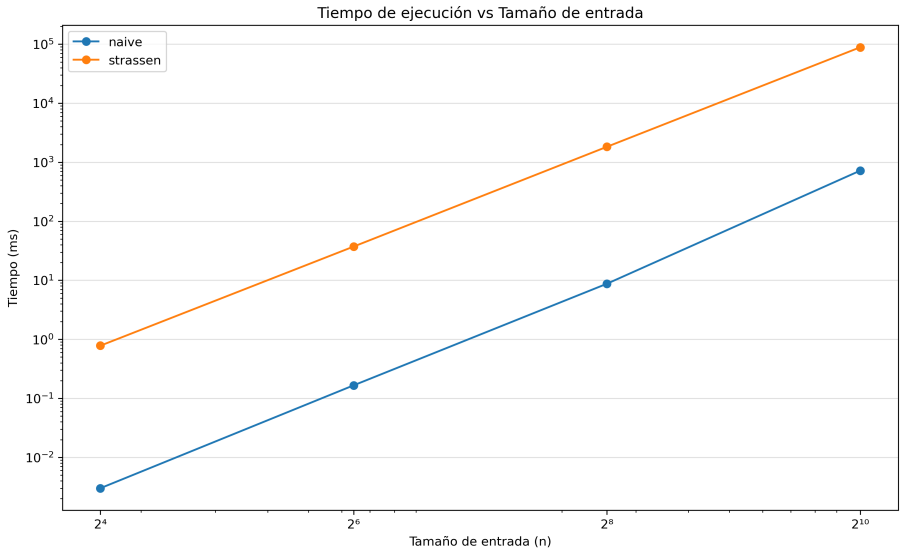

Figura 3: Comparación de tiempo de ejecución entre algoritmos de multiplicación de matrices.

Multiplicación de Matrices - Análisis de memoria

El consumo de memoria en la multiplicación de matrices evidencia claramente el *trade-off* (intercambio) entre la optimización del tiempo de ejecución y el uso de recursos espaciales del sistema.

- **Algoritmo Naive:** La implementación tradicional destaca por su alta eficiencia espacial. Al realizar las operaciones y acumulaciones iterativas leyendo directamente las estructuras de las matrices originales y escribiendo en la matriz de resultados, no requiere la creación de estructuras de datos auxiliares. Su consumo de memoria dinámica extra es prácticamente nulo, manteniendo un *peak* de memoria (RSS) estable y no creciente con una eficiencia espacial de $O(1)$.

- **Algoritmo de Strassen:** Strassen presenta un consumo de memoria significativamente mayor al operar recursivamente subdividiendo las matrices. El procedimiento exige la asignación explícita de numerosas sub-matrices en cada nivel de la recursión para almacenar adiciones y sustracciones intermedias. Este proceso eleva rápidamente el *peak* de memoria, generando una eficiencia espacial adicional de $O(n^2)$, lo que lo hace considerablemente más exigente para el sistema operativo en comparación al otro algoritmo.

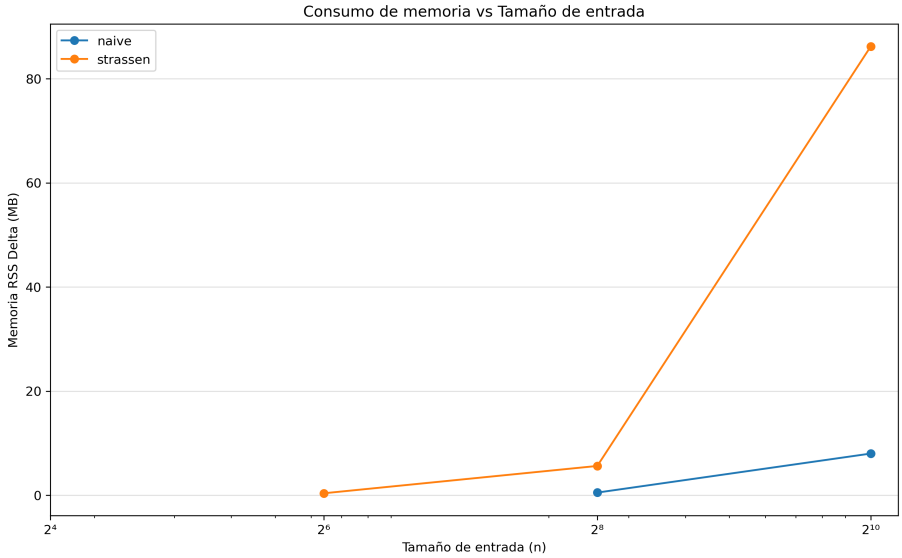

Figura 4: Comparación de uso de memoria entre algoritmos de multiplicación de matrices.

Como se observa en la figura, para tamaños inferiores a 2^6 no se aprecia un consumo de memoria graficado para `Strassen`, lo que se explica por su uso aún muy reducido de memoria en esos casos. Sin embargo, a medida que aumenta el tamaño de las matrices, su consumo de memoria se vuelve cada vez más evidente, tal como era esperable. Por otro lado, en `Naive` recién a partir de tamaños cercanos a 2^8 se comienza a notar un incremento en el consumo de memoria, mientras que para tamaños menores este resulta prácticamente nulo. De esta forma, se aprecia que `Naive` requiere una cantidad de memoria considerablemente menor que `Strassen`, lo cual es coherente con la naturaleza de ambos algoritmos y con la forma en que fueron implementados. (fig. 11)

3. Conclusiones

En conclusión, los resultados experimentales muestran que la notación asintótica permite anticipar correctamente la tendencia general de los algoritmos, pero no explica por sí sola su comportamiento real. En ordenamiento, `Mergesort` y `Quicksort` presentaron un rendimiento estable y cercano a lo esperado, mientras que `std::sort` evidenció una mayor sensibilidad a la implementación y a la medición de la memoria. En multiplicación de matrices, `Naive` resultó más conveniente para tamaños pequeños y medianos por su simplicidad y bajo consumo de memoria, mientras que `Strassen` mostró el costo asociado a su implementación recursiva, especialmente en uso de memoria. En conjunto, este trabajo permite concluir que la eficiencia práctica de un algoritmo no depende únicamente de su complejidad teórica, sino también de factores como la implementación, el tamaño real de la entrada y, adentrando mucho más en este análisis, el método de medición de memoria que pudo ser evidenciado como un factor gatillante a obtener resultados que difieren a lo teórico.

4. Apéndice

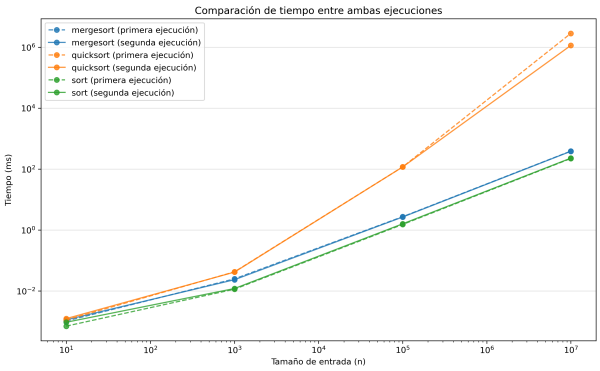

Figura 5: Comparación de tiempos de ejecución entre algoritmos de ordenamiento.

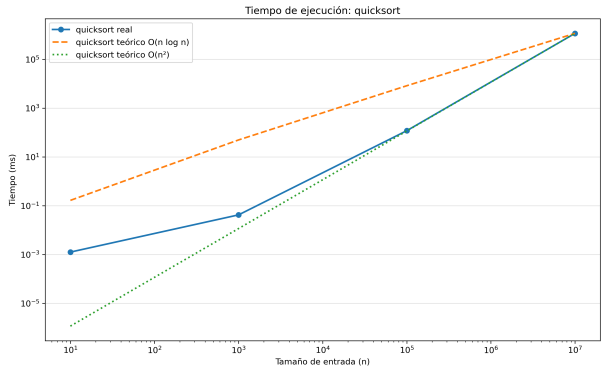

Figura 6: Rendimiento de Quicksort comparado a rendimientos teóricos.

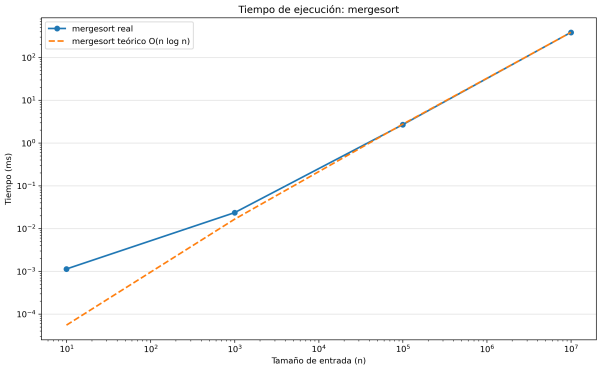

Figura 7: Rendimiento de Mergesort comparado a rendimientos teóricos.

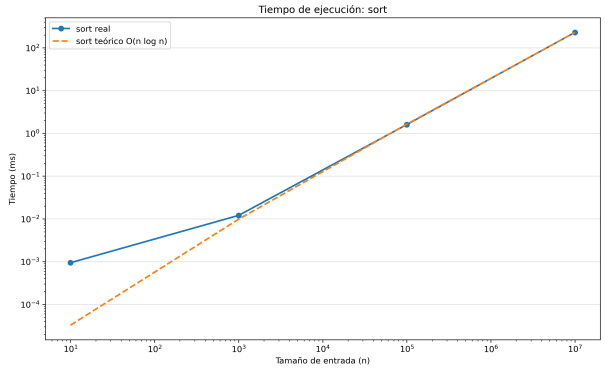

Figura 8: Rendimiento de Sort comparado a rendimientos teóricos.

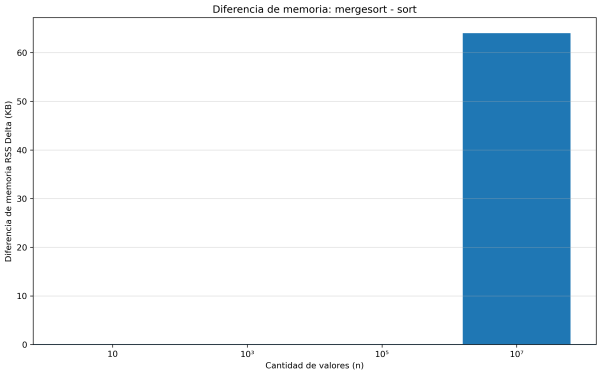

Figura 9: Diferencia de consumos de memoria entre Mergesort y Sort en KB.

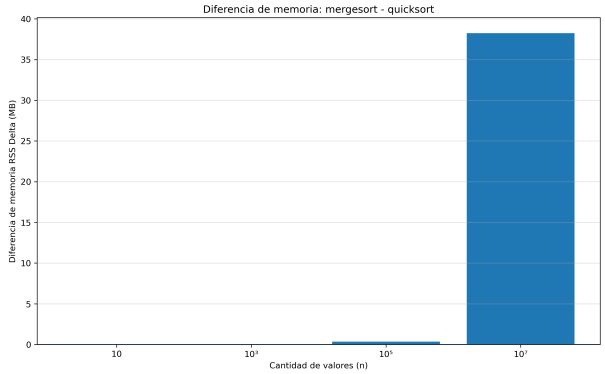

Figura 10: Diferencia de consumos de memoria entre Mergesort y Quicksort en MB.

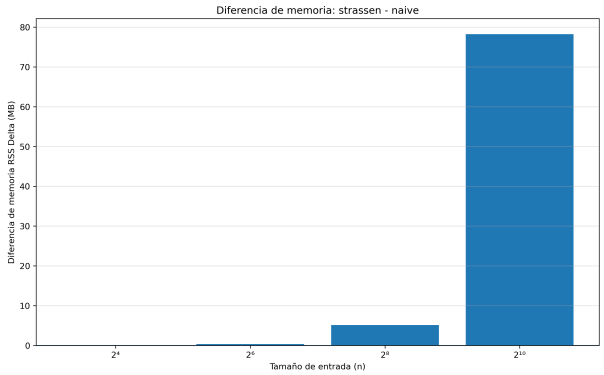

Figura 11: Diferencia de consumos de memoria entre Strassen y Naive en MB.

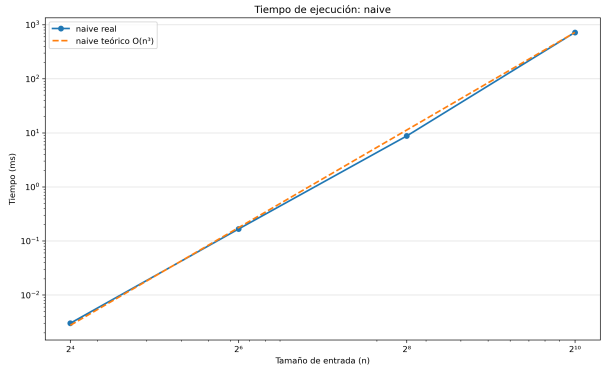

Figura 12: Rendimiento de Naive comparado a rendimientos teóricos.

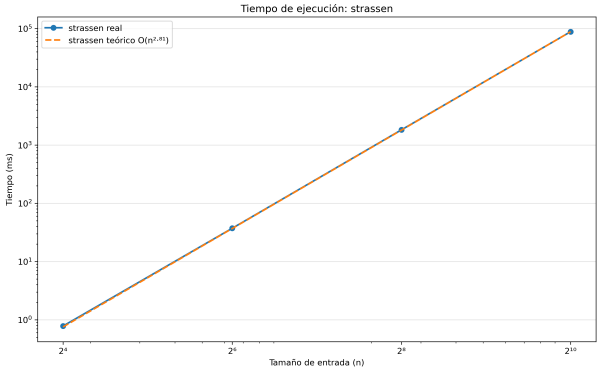

Figura 13: Rendimiento de Strassen comparado a rendimientos teóricos.

Referencias

[1] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. MIT Press, 2009.

[2] Geeks for Geeks. *Merge Sort*. https://www.geeksforgeeks.org/dsa/merge-sort/. 2025.

[3] Geeks for Geeks. *Quick Sort*. https://www.geeksforgeeks.org/cpp/cpp-program-for-quicksort/. 2025.

[4] Geeks for Geeks. *Strassen's Matrix Multiplication*. https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication/. 2025.

[5] ISO/IEC. *ISO/IEC 14882:2020: Programming languages — C++*. Cláusula 11.4.4.2. International Organization for Standardization. 2020.

[6] V. Strassen. «Gaussian elimination is not optimal». En: *Numerische Mathematik* 13.4 (1969), págs. 354-356.